

Software Design Specification (SDS)

Agile System Design & Implementation Readiness Document

Course: PRJ566

Status: FINAL LOCKED VERSION – S26 | Version: v1.0 | Last Updated: 2026-08-12

Section (NAA / NBB / etc.): NCC

Project Title: FitCord

Team Number: 2

Team Members: Julie, Bilal, Renz

Instructor / Faculty Reviewer: Beshid Behkamal

Date: May 25th, 2026

Github Repo: [FitCord Repo](#)

Prototype Link: [Figma Link](#)

This SDS is a living artifact that evolves across sprints. Not all sections are expected to be complete at the beginning of the semester.

Executive Summary

Background

Description

Company Value Add

End-User Value Add

Scope: What is Included / What is Not Included

Justification

Table of Contents

[Executive Summary](#)

[1. Introduction \(Week 1-2\)](#)

[1.1 Document Authors](#)

[1.2 Document Revision History](#)

[1.3 Document Purpose](#)

[1.4 Audience](#)

[1.5 Group Agreement](#)

[1.6 File Naming Convention \(Required\)](#)

[2. Project Proposal & Scope \(Week 2 / Sprint 1\)](#)

This section is derived from the approved Business Proposal. Content should be refined, clarified, and aligned with system design decisions. Do not rewrite the proposal - focus on improving precision, scope, and technical clarity.

[2.1 Project Background](#)

[2.2 Problem Statement](#)

[2.3 Product Vision](#)

[2.4 Stakeholders and Users](#)

[2.5 Project Scope](#)

[2.6 System Risks](#)

[2.7 Operating Environment](#)

[3. System Behavior & Scenarios \(Sprint 1-2\)](#)

[3.1 Preliminary Product Backlog](#)

[3.2 Functional Requirements](#)

[3.3 Non-Functional Requirements](#)

[3.4 Data Flow Diagrams \(DFDs\)](#)

[3.5 Business Rules](#)

[3.6 Refined User Stories and Use Case Scenarios](#)

[3.7 Scenario Flows & UI Mapping](#)

[4. Sprint 3 Design Artifacts](#)

[4.1 UI/UX Design & Prototype](#)

[4.2 Domain Model](#)

[5. Data Design \(Sprint 3\)](#)

[5.1 RDBMS Design: ERD](#)

[5.2 RDBMS Design: Data Dictionary](#)

[5.3 NoSQL Design Option](#)

[6. Agile Planning Overview \(Sprint 4\)](#)

[6.1 System Setup & Environment Readiness](#)

[6.2 GitHub & Project Setup](#)

[6.3 Vertical Slice Planning](#)

[7. Product Backlog & Sprint Readiness \(Sprint 4\)](#)

[7.1 Product Backlog](#)

[7.2 Sprint 1 Candidates](#)

[8. Client / Faculty Sign-off](#)

The Table of Contents above is linked to each section. Use Ctrl+Click in Word to navigate.

1. Introduction (Week 1-2)

1.1 Document Authors

Add or remove rows as needed. The number of rows in this template is not a required limit.

Name	Role / Responsibility	Email
Bilal Umar	Developer	bumar@myseneca.ca
Julie Rundell	Organizer, Developer	Jrundell@myseneca.ca
Renz Navarete	Team Lead, Developer	rnavarrete2@myseneca.ca

1.2 Document Revision History

Add or remove rows as needed. The number of rows in this template is not a required limit.

Week / Sprint	Date	Revisions Made
Week 2	2026-05-20	Added approved proposal, scope, and initial stakeholders.
Week 3	2026-05-25	Transferred over and reconfigured material from initial documents to SDS sheet
Week 4	2026-05-27	Completed remaining sections for Sprint 1
Week 5	2026-06-07	Completed revisions of SDS document based on professor feedback
Week 6	2026-11	Added user stories, functional and non-functional requirements, business rules, refined stories and scenarios, and scenario flows and UI mapping
Sprint 3	2026-07-26	Created visual prototypes to match flow of logic, created database structure
Sprint 4	2026-08-12	Setup development environment, groomed user stories for first sprint development, established product backlog

1.3 Document Purpose

This document defines the system design for the proposed solution using an Agile approach. It includes requirements, user scenarios, system behavior, UI/UX design, data design, and implementation readiness artifacts. The document evolves across sprints and prepares the system for implementation in PRJ666.

1.4 Audience

Identify the intended readers such as faculty, team members, stakeholders, and future PRJ666 developers.

- Team Members
- Professor (Behshid Behkamal)

1.5 Group Agreement

This section defines the team's working agreement, responsibilities, communication expectations, and overall commitment throughout PRJ566 and PRJ666.

Team Overview

Add or remove rows as needed. The number of rows in this template is not a required limit.

Item	Team Response
Team Number	Team 02
Project Title	FitCord
Project Time Frame	PRJ566 (Planning & Design) to PRJ666 (Implementation)

Team Members

Add or remove rows as needed. The number of rows in this template is not a required limit.

Name	Student ID	Email	Signature
Bilal Umar	185499233	Bumar@myseneca.ca	B.U
Julie Rundell	118377209	Jrundell@myseneca.ca	J.R.
Renz Navarete	121702237	rnavarrete2@myseneca.ca	R.N.

Team Leadership

Describe how leadership responsibilities will be managed within the team.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Leadership Area	Team Response
Coordination Approach	Leadership responsibilities may rotate depending on sprint goals and project needs. The team lead is responsible for coordination, communication, and ensuring sprint deliverables are completed on time.
Democratic Approach	The team will continue to work together and come to common consensus on project matters. Team lead will help to guide the process while listening to insights and ideas of project members.

Team Functions / Roles

Add or remove rows as needed. The number of rows in this template is not a required limit.

Team Member	Primary Role	Secondary Responsibilities
Developer	Develop and implement systems	Suggest new features and functionality Check scope creep
Organizer	Help create structure for coherent development	Ensure team lead's vision is being accomplished
Researcher	Find primary sources for documentation	Challenge assumptions made by other team members with evidence
Team Lead	Ensure work is on schedule and assigned properly	Orientate roles based on strengths and weaknesses

Team Meetings

Describe the expected meeting schedule and communication approach.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Meeting Type	Frequency / Timing	Platform / Location	Notes
Team Meeting	Thursday Afternoons	Microsoft Teams / in person as needed	Additional meetings may be scheduled during sprint reviews or major milestones.

Team Strategy for Managing Issues and Conflict Resolution

Describe how the team plans to handle communication issues, missed deadlines, uneven workload distribution, and team conflicts.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Issue Type	Team Strategy	Escalation / Notes
Missed deadline or uneven workload	Discuss internally, document the issue, and reassign or adjust tasks if needed.	Persistent issues should be escalated to the professor when appropriate.
Lack of member availability or participation	Communicate schedule or availability changes early	Repeated inactivity without communication will be escalated to professor
Unclear tasks/ responsibilities	Define tasks/ responsibilities at the beginning of each sprint	Confusion regarding responsibilities should be discussed with team

Team Commitment

The undersigned members agree to work together on the project throughout PRJ566 and continue into PRJ666 during the following semester. Team members recognize that both individually and collectively they are responsible for the quality, professionalism, and timely completion of all project deliverables.

Important Note: Team agreements may be updated if approved by the professor and agreed upon by all team members.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Name	Date	Signature
Julie Rundell	2026-05-25	JR
Bilal Umar	2026-05-25	BU
Renz Navarrete	2026-05-27	RN

1.6 File Naming Convention (Required)

To ensure consistency and proper tracking across all teams and submissions, the following file naming convention must be used for all SDS documents:

Format:

Team##_PRJ566_N##_SDS_S26_WK##

Example:

Team03_PRJ566_NAA_SDS_S26_WK05

Naming Components:

- **Team##** → Team number (e.g., Team01, Team02)
- **PRJ566** → Course code
- **N##** → Section identifier (e.g., NAA, NBB)
- **SDS** → Document type
- **S26** → Semester (Summer 2026)
- **WK##** → Week or Sprint number of the submission

Important Notes:

- All submissions must follow this naming convention exactly
- Update the **WK##** value for each submission or major update
- Do not use spaces or additional characters in the file name
- Consistent naming ensures proper version tracking and evaluation

2. Project Proposal & Scope (Week 2 / Sprint 1)

2.1 Project Background

Many fitness apps focus on keeping users inside their own platforms, requiring users to build new routines and communities from scratch. While these apps help track workouts and progress, long-term consistency often depends on social interaction and accountability rather than tracking alone.

For Gen Z, online communities are already a major part of everyday life and act as a digital “third space” in the modern world. Platforms like Discord are places where users regularly interact with their friends and communities. However, most fitness apps separate fitness from these existing spaces, creating extra friction, and making consistency harder to maintain. Fitcord addresses this by bringing fitness into communities users already use and engage with daily.

2.2 Problem Statement

The problem of fitness apps operating separately from users' existing online communities

Affects Gen Z discord users who rely on social interaction for motivation and accountability

The impact of which is reduced workout consistency and long-term engagement.

A successful solution would integrate fitness application functionality into pre-existing, online social communities.

2.3 Product Vision

Fitcord is for Gen Z users who want to stay consistent with their fitness goals while remaining active in their existing online communities

Who need motivation, accountability, and seamless integration between fitness tracking while maintaining their existing social bubbles.

Fitcord is a social-integrated fitness platform

That connects workout tracking, challenges, and progress sharing directly into Discord communities.

Unlike traditional fitness apps that require users to build new habits and isolated communities within a separate platform

Fitcord brings fitness into existing social ecosystems, enabling consistency through familiar communities, shared goals, and continuous social reinforcement.

2.4 Stakeholders and Users

Add or remove rows as needed. The number of rows in this template is not a required limit.

Stakeholder / User Group	Internal or External	Needs / Interests	Notes
Discord Users	External	Fitness application that doesn't compete for attention or isolate them from main social platform	Primary Target
Community Organizers	External	Community building elements that integrate into wider, existing ecosystems than other isolated fitness apps	Secondary Target
Gyms and Rec Centers	External	Provides exposure to their centers via a community driven application with a map function	Secondary Target

* Based on research, Discord Users broadly captures our Gen-Z demographic based on weekly use statistics

2.5 Project Scope

Add or remove rows as needed. The number of rows in this template is not a required limit.

Included in Scope	Not Included in Scope	Rationale
Discord bot for workout logging and interaction	High-scale distributed system	High user count and traffic is not expected
Mobile app for tracking and sharing workouts and progress	3 rd party integration (outside discord), AI, etc.	Not needed for core processes, adding extended integration time for smaller ROI
App functionality for community discovery	Individual advertising services	The app's discoverability functionality is for a community-oriented experience
Real-time syncing between the app and Discord	Additional app support	The discord and app alone are the core scope of our user experience and additional app integration may/will cause bloat and scope creep.
Backend database and user management systems	Guest App Usage	App software and bot integration will require some level of registration to manage cross-platform data sharing.

2.6 System Risks

Add or remove rows as needed. The number of rows in this template is not a required limit.

Risk ID	Risk Description	Impact	Mitigation
R-01	Team may underestimate integration effort.	Medium	Use vertical slice to validate UI, logic, and data early.
R-02	Security vulnerabilities involving Discord integration or backend APIs	High	Use secure authentication practices, store sensitive keys in environment variables, and restrict unauthorized data access
R-03	Unforeseen data loss/corruption	High	Perform regular database backups
R-04	Workout data may become inconsistent mobile app and discord bot	High	Log synchronization failures for debugging and recovery
R-05	Users may find onboarding or Discord setup confusing	Medium	Provide clear instructions for Discord server integration

2.7 Operating Environment

Add or remove rows as needed. The number of rows in this template is not a required limit.

Environment Area	Description / Assumption
Discord Bot	Runs and integrates on Discord servers
Mobile App	Allows for simple fitness tracking and modular recording
Database (NoSQL MongoDB Database)	Stores our user data

3. System Behavior & Scenarios (Sprint 1-2)

This section captures the reviewed Sprint 2 items in the same sequence as the deliverable schedule: preliminary backlog, FRs, NFRs, DFDs, business rules, and refined stories/scenarios.

3.1 Preliminary Product Backlog

The preliminary backlog includes high-level user stories with descriptions only. Detailed grooming is required only for Sprint 1 candidates in Section 7.2.

Add or remove rows as needed. The number of rows in this template is not a required limit.

User Story ID	Description	Priority	Status / Notes
US-01	As a Discord user, I want to log my workout directly in Discord so that I don't have to leave my community to track fitness.	High	Sprint 1 candidate
US-02	As a Discord user, I want to join fitness challenges with my server members so that I stay accountable through my existing social circle.	High	Sprint 1 candidate
US-03	As a Discord User, I want to track my workouts and view my progress so that I can monitor my fitness over time.	High	Sprint 1 candidate
US-04	As a Discord user, I want my workout logs synced between the mobile app and Discord bot so that my data is consistent across both platforms.	High	Sprint 2 candidate
US-05	As a Community Organizer, I want to create a public fitness community discoverable within the app so that I can grow my group beyond my existing Discord server or community in real life.	Medium	Sprint 2 candidate
US-06	As a Discord user, I want to see a leaderboard of my server's workout activity so that I'm motivated by friendly competition.	Medium	Sprint 2 candidate
US-07	As a Discord User, I want to find nearby gyms and rec centers on a map so that I can discover places to work out near me.	Low	Future backlog
US-08	As a gym or rec center, I want my location listed in the app's community map so that I gain exposure to fitness-oriented Gen Z users.	Low	Future backlog

3.2 Functional Requirements

Functional requirements should be derived from user stories and scenarios and should remain traceable.

Add or remove rows as needed. The number of rows in this template is not a required limit.

FR ID	Functional Requirement Description	Related User Story / Scenario	Priority
FR-01	The system shall allow users to authenticate via Discord OAuth to link their Discord account to FitCord.	US-06	High
FR-02	The system shall allow users to log a workout session through a Discord bot command.	US-01	High
FR-03	The system shall allow users to log a workout session through the mobile app.	US-03	High
FR-04	The system shall sync workout data between the mobile app and Discord bot in real time.	US-04	High
FR-05	The system shall allow Discord server members to join shared fitness challenges.	US-02	High
FR-06	The system shall display a leaderboard of workout activity for users within the same Discord server.	US-07	Medium
FR-07	The system shall allow users to view their personal workout history and progress metrics in the mobile app.	US-03	High
FR-08	The system shall allow fitness community builders to create and publish a discoverable public community profile within the app.	US-05	Medium
FR-09	The system shall display a map of nearby gyms and rec centers based on the user's location.	US-08, US-09	Low

3.3 Non-Functional Requirements

Include important constraints such as performance, security, usability, availability, privacy, and platform compatibility.

Add or remove rows as needed. The number of rows in this template is not a required limit.

NFR ID	Non-Functional Requirement Description	Category	Measurement / Criteria
--------	--	----------	------------------------

NFR-01	Bot commands and mobile app core screens shall respond within 2 seconds under normal load.	Performance	Response/load time $\leq$ 2 seconds
NFR-02	Workout data shall sync between mobile and Discord within 5 seconds of submission.	Performance	Sync latency $\leq$ 5 seconds
NFR-03	All authentication shall use Discord OAuth 2.0; no passwords or secrets stored in plaintext or version control.	Security	OAuth 2.0 compliance; zero exposed credentials in repo
NFR-04	The app shall be functional on iOS 16+ and Android 12+.	Compatibility	Tested on both platforms before each release
NFR-05	The bot onboarding flow shall be completable by a non-technical server admin within 5 minutes.	Usability	Validated via usability testing
NFR-06	All UI shall meet WCAG 2.1 AA contrast minimums and follow a consistent FitCord design system.	Accessibility / UX	Contrast ratio $\geq$ 4.5:1; design system applied across all screens
NFR-07	The system shall maintain 99% uptime and log all sync failures with enough context for recovery.	Reliability / Availability	$\geq$ 99% uptime monthly; no silent data loss on failure

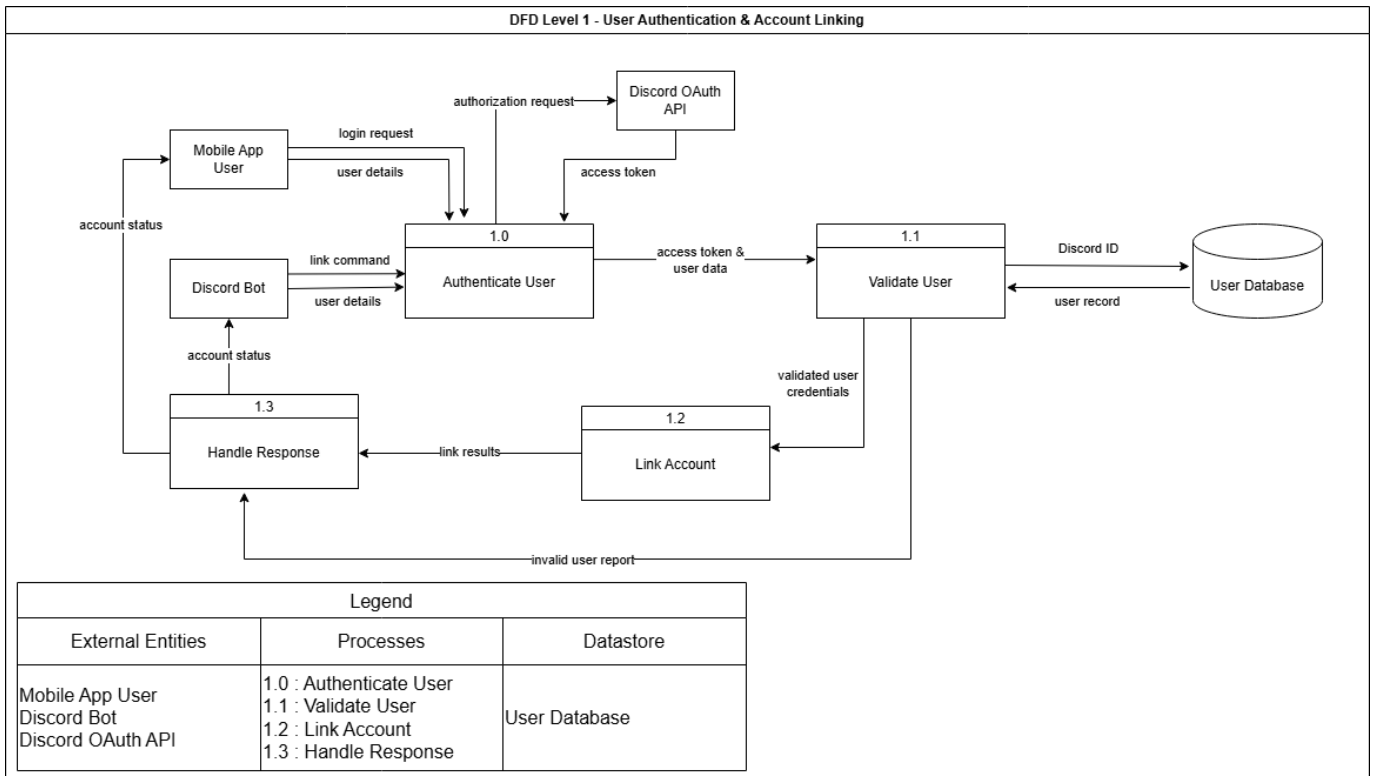
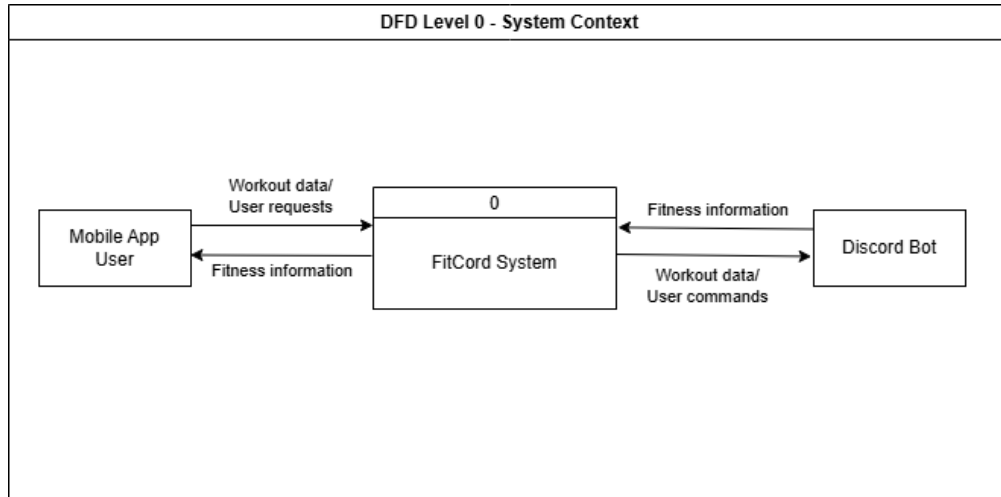
3.4 Data Flow Diagrams (DFDs)

Include DFDs for key system flows only. Activity diagrams are not required in this template.

Add or remove rows as needed. The number of rows in this template is not a required limit.

DFD ID	Level / Name	Related Scenario(s)	Brief Description
DFD-01	Level 0 — System Context	All	Shows FitCord as a single system with three external actors: the user, the Discord bot, and the backend database. Establishes the boundary of the system.
DFD-02	Level 1 — User Authentication & Account Linking	US-06	User initiates Discord OAuth → FitCord receives token → backend validates and stores linked account. Entry point for all other flows.
DFD-03	Level 1 — Cross-Platform Sync	US-04	Workout submitted on either platform → sync service reconciles → updated state pushed back to both mobile and Discord bot. Directly maps to R-04.
DFD-04	Level 1 — Challenge & Leaderboard Flow	US-02, US-07	User joins or completes a challenge → backend updates challenge state → leaderboard recalculated →

			bot pushes updated standings to Discord server.
DFD-05	Level 1 — Community Discovery	US-05, US-08	App queries community/gym data → backend queries database → results returned and rendered on map or community list.



3.5 Business Rules

List only critical business rules that affect system behavior, validation, workflow, or decision logic.

Add or remove rows as needed. The number of rows in this template is not a required limit.

BR ID	Description	Related Scenario / User Story	Related UI / System Area
BR-01	A user must have a linked Discord account before logging any workout through either the bot or the mobile app.	US-06	Onboarding flow, bot command handler
BR-02	A workout log submitted via bot and an identical log submitted via mobile within the same session shall be treated as one entry, not two.	US-04	Sync service, workout history
BR-03	A user may only join a server challenge if they are a member of the associated Discord server.	US-02	Challenge join screen, bot validation
BR-04	A leaderboard shall only display users who have opted into sharing their workout activity within that server.	US-07	Leaderboard screen, privacy settings
BR-05	A Discord server admin must authorize the FitCord bot before any fitness features become available to server members.	US-10	Bot setup flow, server permissions
BR-06	Workout data shall not be deleted from the backend if a sync failure occurs; the failure shall be logged and retried.	US-04	Sync service, error logging
BR-07	A community profile shall only be publicly discoverable after the creator has completed account linking and submitted at minimum a name and description.	US-05	Community creation form, discovery feed

3.6 Refined User Stories and Use Case Scenarios

Use this section to refine the most important stories and describe key end-to-end user interactions.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Scenario ID	Actor / Persona	Trigger	Main Flow	Alternative / Exception Flow
US-01	Discord User	User inputs workout logging command	Enter command -> Bot validates input -> Bot sends data to backend -> Bot confirms successful log	Invalid workout data is entered. The bot requests re-input
US-02	Discord User	User selects to join a challenge	User views challenge -> User joins challenge -> System verifies eligibility -> System	User attempts to join completed or deleted challenge. Join request is

			adds user to challenge record -> System sends confirmation	rejected.
US-03	Discord User	User logs workout	User inputs workout details -> App validates info -> App sends details to backend -> Backend stores details -> Updated history is displayed	Network failure prevents submission
US-04	FitCord System	New workout is created	Workout details is received -> Backend stores details -> Updated data is sent to app and Discord bot -> Both platforms display updated history	Synchronization fails and schedules retry
US-05	Discord User	User opens the community map	Find community pin that aligns with interests -> Click on pin -> Click discord invite link	Discord invite link is expired, invite rejected

3.7 Scenario Flows & UI Mapping

Map user stories and scenarios to UI screens or prototype flows. This replaces the previous activity diagram requirement.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Scenario ID	Related User Story	UI Screen / Mockup Reference	Flow Notes
SC-01	US-06	UI-01 Onboarding / Account Linking	User opens app → registers information → Given ID is bounced back when they first join a server for verification
SC-02	US-10	UI-02 Bot Setup Flow	Admin navigates to bot invite link → authorizes FitCord bot in their server → bot posts setup confirmation in server channel.
SC-03	US-01	UI-03 Discord Bot — Workout Log	User types log command in Discord → bot prompts for workout details → user confirms → bot posts workout summary to channel.
SC-04	US-03	UI-04 Mobile — Workout Log Screen	User opens app → selects log workout → enters exercise, sets, reps/duration → submits → confirmation shown, data saved.
SC-05	US-04	UI-04, UI-03	Workout logged on mobile → sync service fires → bot posts update to linked Discord server automatically. Reverse path also applies.
SC-06	US-02	UI-05 Challenge Screen	User views active server challenges → selects one → joins → progress tracked per workout log → bot posts updates to server.

SC-07	US-07	UI-06 Leaderboard Screen	User opens leaderboard tab → sees ranked server members by workout activity → bot mirrors top standings in Discord channel.
-------	-------	--------------------------	---

4. Sprint 3 Design Artifacts

4.1 UI/UX Design & Prototype

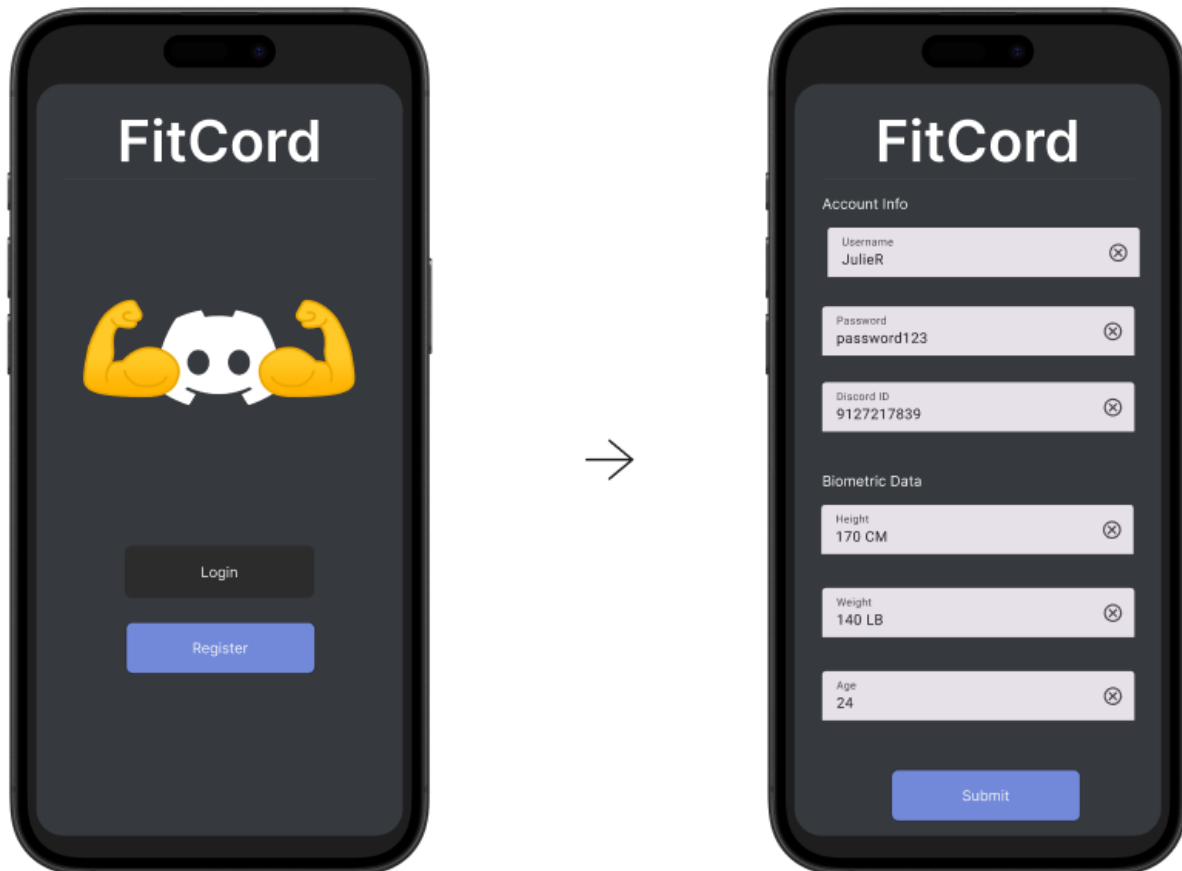
Develop a scenario-driven prototype based on personas, user stories, and use case scenarios. The prototype should demonstrate end-to-end flows rather than isolated screens.

[PROTOTYPE LINK - FIGMA](#)

Add or remove rows as needed. The number of rows in this template is not a required limit.

Prototype Area	Related Scenario / User Story	Screen / Flow Reference	Notes
Login/Register	SC-01	UI-01	Shows user registration and login login
Community Finder	US-05	UI-02	Shows community map, pin system, and joining prompt
Start Workout	SC-04	UI-05 to UI-06	Shows workout activity, breakdown of workouts, and logging technique
Discord Challenge	SC-06	UI-03 to UI-04	Shows the discord bot system on discords platform

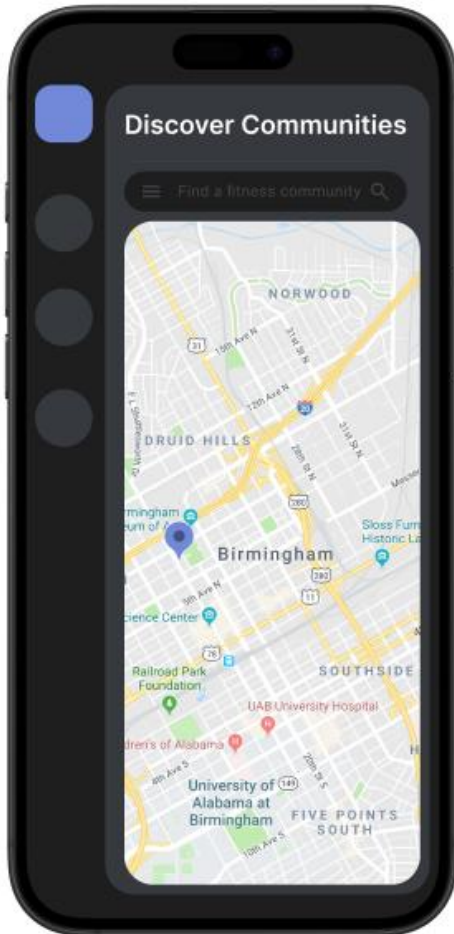
UI-01



UI-02

Map Tab

- Filters to address and is traversable
- Publicly listed communities pop up on the map with info and discord server invite when clicked on
- Can be further filtered based on server tags (Cycling, Lifting, Running, etc.)

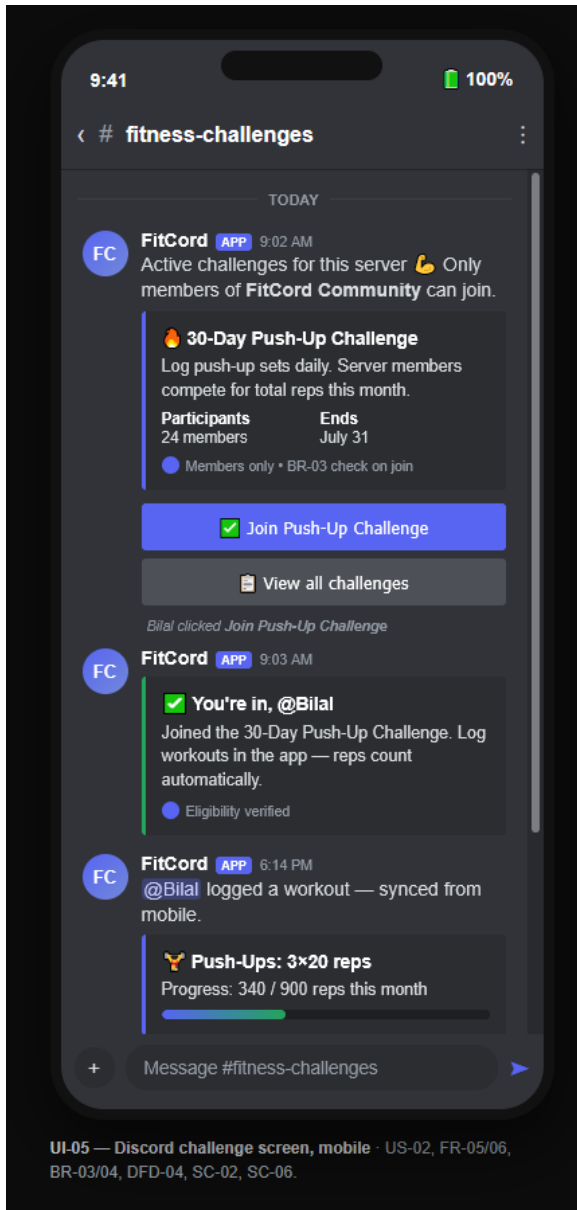


Popup

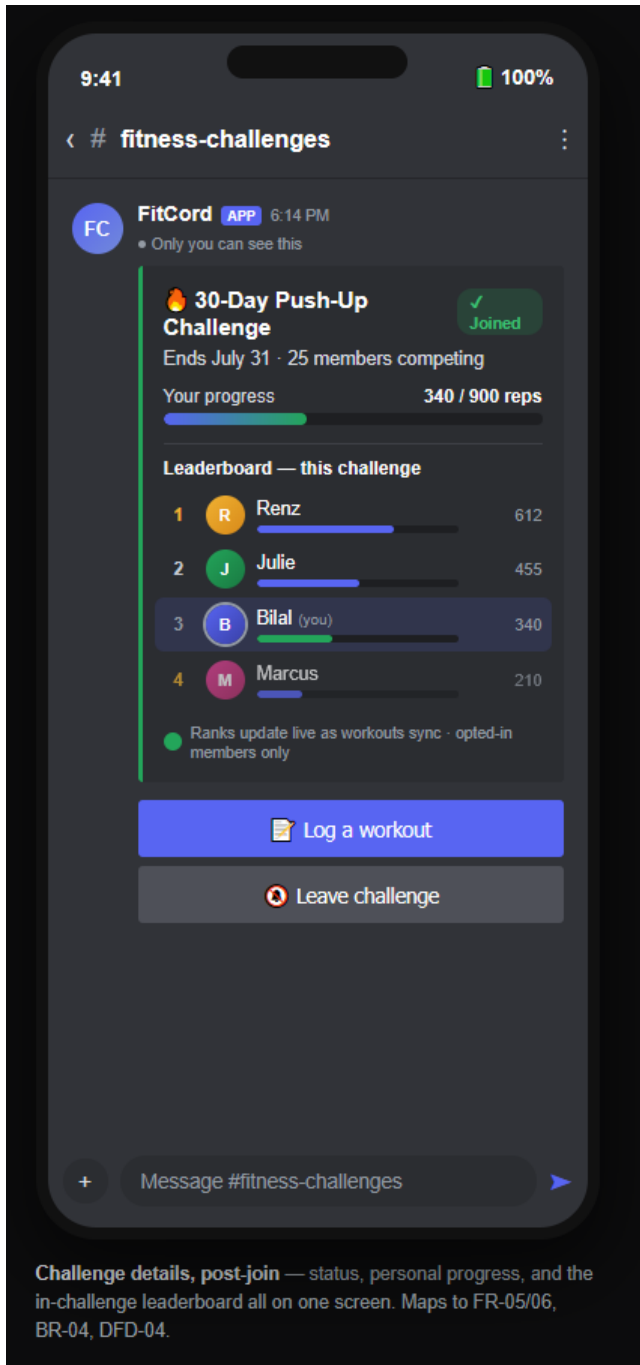
- Pops up when a node is clicked on the map
- Gives the title, description, photo (optional), discord link, and associated tags



UI-03

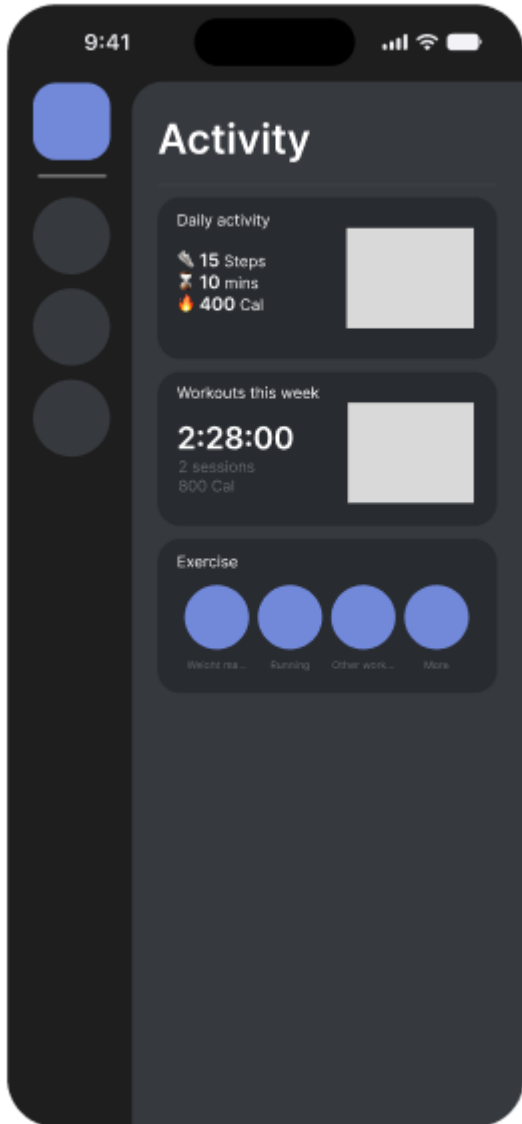


UI-04



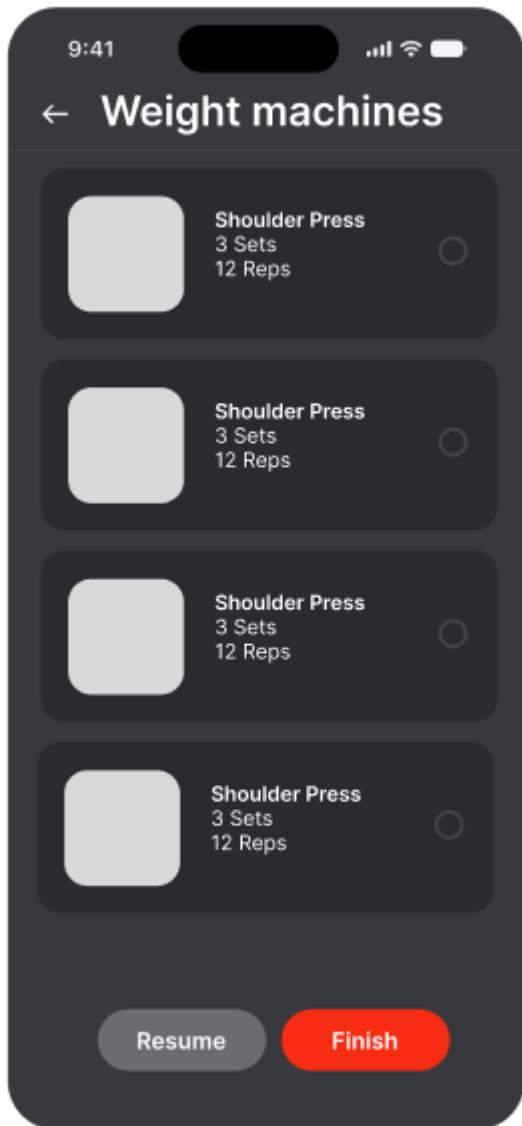
UI-05

Activity View - iPhone



UI-05

Activity View - iPhone



4.2 Domain Model

Insert the Domain Class Diagram and briefly explain the major classes/entities and relationships.

Add or remove rows as needed. The number of rows in this template is not a required limit.

We utilize NoSQL

Class / Entity	Key Attributes	Relationships	Notes

5. Data Design (Sprint 3)

Choose the database design approach that best fits the system. If both RDBMS and NoSQL are used, complete both relevant subsections.

5.1 RDBMS Design: ERD

Insert the Entity Relationship Diagram (ERD) and explain the major entities and relationships.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Entity	Primary Key	Relationships	Notes
Booking	booking_id	Many bookings belong to one user; one booking reserves one equipment item.	Supports the main booking scenario.

5.2 RDBMS Design: Data Dictionary

Complete this table for the key entities and fields shown in the ERD.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Entity / Table	Field Name	Data Type	Required?	Description / Validation Rule
Booking	booking_id	INT	Yes	Unique identifier for each booking record.

5.3 NoSQL Design Option

Use this section if the solution uses NoSQL. Define data models and provide corresponding sample documents.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Collection / Document Type	Purpose	Key Fields	Sample Document / Structure Notes
Exercises	Stores exercises of various types for the user to use and reuse	_id , Name , Type , Reps Sets, Distance, Duration, ExpectedCaloriesBurned, Equipment , Owner	{ "_id": "EX001", "Name": "Push Ups", "Type": "Strength", "Description": "Standard bodyweight push ups.", "Reps": 15, "Sets": 4, "Distance": 0.0, "Duration": 10, "ExpectedCaloriesBurned": 80, "Equipment": [], "Owner": { ... } }
Users	Registers them to the app	_id, Username, Email, Password, Communities, Owns, DiscordID, ValidID, CreationDate, Height, Weight, Age	{ "_id": "USR001", " Username ": "JulieR", "Email": "julie@example.com", "Password": "password123", "Communities": [], "Owns": [], "DiscordId": 181727261811, "ValidId": true, "CreationDate": "2026-07-20" "Height": 170, "Weight": 145, "Age": 23, }
Communities	Enables joinable environments and sociability	_id, Name, Description, Image, InviteLink, Owner, Associates, Location, Visible, Category	{ "_id": "COM001", "Name": "Toronto Runners", "Description": "A community for runners of all skill levels in the Greater Toronto Area.", "Image": "toronto- runners.png", "InviteLink": "https://discord.gg/torunners", "Owner": "USR001", "Associates": [{ ... },], "Location": [43.6532, - 79.3832], "Visible": true, "Category": "Running" }
Challenges	Encourages consistency and participation	_id, ChallengeName, ChallengeDesc, ChallengeExercise, StartDate, EndDate, Community, Participants, ParticipantExercises, IsComplete, Owner	{ "_id": "CH001", "ChallengeName": "100 Push- Up Challenge", "ChallengeDesc": "Complete 100 push-ups every day for one week.", "ChallengeExercise": { ... }, "StartDate": "2026-07-20", "EndDate": "2026-07-27", "Community": { ... }, "Participants": [{ ... }], " ParticipantExercises": [{ ... }], "IsComplete": false, "Owner": { ... } }

Important: Database design must support at least one complete end-to-end user scenario demonstrated in the prototype.

6. Agile Planning Overview (Sprint 4)

This section captures the system's readiness for implementation in PRJ666. It focuses on environment setup, repository structure, and the initial vertical slice required to begin development.

6.1 System Setup & Environment Readiness

Add or remove rows as needed. The number of rows in this template is not a required limit.

Item	Description	Status / Notes
Development Environment	GIT repo established and access granted to developers	Ready for PRJ666 implementation.
MongoDB Setup	Project can perform CRUD operations on data	Created – Mock Data Installed
Discord Setup	Developers are familiar and setup on Discord	Ready for PRJ666 implementation.

6.2 GitHub & Project Setup

Add or remove rows as needed. The number of rows in this template is not a required limit.

Item	Description	Status / Notes
GitHub Repository Link	Repository created with README, project structure, and initial setup instructions.	Ready / link included by team.
MongoDB	Database for storing data	Created – Mock Data Installed

6.3 Vertical Slice Planning

A vertical slice represents one complete end-to-end feature that connects the UI, application logic, and data layers. In PRJ566, the vertical slice is designed, validated, and demonstrated conceptually, ensuring readiness for PRJ666 implementation.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Feature	UI Component(s)	Application Logic / Service	Data Layer	Status / Notes
Joining Community	Community Map	Render community data and proper invite link	Community Info	In Progress
Logging Workout	Workout Tab	Render workout screen, take in data, store it	Workout Data	In Progress
Joining Challenge	Discord Bot - Challenges	Provide challenge data, take in data, store it	Challenges Data	In Progress

7. Product Backlog & Sprint Readiness (Sprint 4)

7.1 Product Backlog

The general product backlog includes high-level user stories with clear descriptions only. Detailed grooming can continue in PRJ666.

Add or remove rows as needed. The number of rows in this template is not a required limit.

User Story ID	Description	Priority	Status / Notes
US-01	As a Discord User, I want to log workouts, so that I can track my progress	High	N/A
US-05	As a Discord User, I want to access the community map, so I can find fitness communities	High	N/A
US-02	As a Discord User, I want to join challenges, so that I can participate in communities	Medium	N/A

7.2 Sprint 1 Candidates

Only Sprint 1 candidate stories must be fully groomed before PRJ666 Sprint 1 planning.

Add or remove rows as needed. The number of rows in this template is not a required limit.

User Story ID	Description	Acceptance Criteria	Testing Scenarios	Notes
US-05	As a student, I want to access the community map so that I can find communities to join.	Given there are joinable communities, when the user selects a pin on the map, then the community's information is displayed.	Test pins appear; Test the appropriate data with discord link is visible; Test hidden communities don't appear.	Candidate for PRJ666 Sprint 1.
US-01	As a Discord User, I want to log workouts, so that I can track my progress	Given the user is logged in, when the user selects the workout area, then the user can log exercises	Workout area loads; Workouts aren't lost when user switches screens; User can save workouts to DB.	Candidate for PRJ666 Sprint 1.

8. Client / Faculty Sign-off

Use this section to confirm final review, scope approval, and readiness for PRJ666 implementation.

Add or remove rows as needed. The number of rows in this template is not a required limit.

Name	Role	Approval / Comments	Date	Signature
Faculty Reviewer	Instructor	Approved for PRJ666 readiness review.	YYYY-MM-DD	